PROYECTO — MONITOR DE PROCESOS CON CONTROL

1. EXPLICACIÓN

El grupo va a construir un monitor de procesos a pantalla completa para la terminal, que además de mostrar el estado de la máquina permita actuar sobre ella. Todos los datos se obtienen consultando directamente la interfaz que el sistema operativo expone sobre sus procesos; el programa no puede usar una herramienta externa para llenar la tabla ni ninguna librería que devuelva la lista de procesos o el porcentaje de CPU ya calculados. El porcentaje de CPU se calcula en el código: se toma el tiempo de CPU acumulado de cada proceso y el tiempo total del sistema en dos refrescos consecutivos, y se divide la diferencia del proceso entre la diferencia del sistema.

Los lenguajes admitidos son C, C++, Go y Rust. El programa debe ser nativo del sistema operativo en el que se ejecuta.

2. QUÉ DEBE HACER

El formato exacto de la pantalla queda a criterio del grupo.

1. Tabla de procesos con una fila por proceso, con las columnas: identificador, proceso padre, usuario, estado, %CPU, %MEM, memoria residente, número de hilos, prioridad, tiempo de CPU y comando. 2. Resumen del sistema en la parte superior: procesos totales, cuántos corren, cuántos están detenidos, cuántos terminados sin recoger, carga promedio y memoria total, usada y disponible.
3. %CPU propio, calculado por diferencia de tiempos entre refrescos a partir de los contadores del proceso y del sistema; coherente con el que reporta el sistema.
4. Ordenamiento por CPU y por memoria: el proceso que más consume queda primero.
5. Vista de árbol de procesos: la jerarquía padre → hijo.
6. Vista de hilos de un proceso, con el estado de cada uno.
7. Histórico de CPU por proceso a lo largo del tiempo.
8. Resaltado de los procesos terminados sin recoger y de los bloqueados esperando entrada/salida. 9. Actuar sobre un proceso: terminar, terminar por la fuerza (con confirmación aparte), suspender y reanudar. Tras terminar, el proceso desaparece; tras suspender, queda detenido; tras reanudar, continúa. 10. Mostrar y cambiar la prioridad de un proceso.
11. TUI a pantalla completa: ocupa toda la terminal y usa su alto y ancho reales (se adapta si la ventana se redimensiona); se redibuja en el mismo lugar en cada refresco, sin parpadeo ni restos de la tabla anterior; distribuye encabezado con el resumen, tabla en el medio y una línea de ayuda/entrada abajo; y al salir restaura la terminal (eco, cursor y prompt normales).
12. Intervalo de refresco configurable desde la interfaz.
13. Cada acción con su tecla, respondiendo en caliente: salir, cambiar el intervalo, ordenar, filtrar por nombre, ver el árbol, ver hilos, actuar sobre un proceso y cambiar prioridad.
14. Confirmación al actuar: indica sobre qué proceso y qué acción se aplicó.

15. Mensaje claro, sin caerse, cuando un proceso ya no existe o no hay permiso para consultarlo.

3. REGLAS DE IMPLEMENTACIÓN

3.1 Reglas de código

Los datos salen de la interfaz del sistema operativo. El programa consulta directamente la vía que expone el sistema para leer el estado de sus procesos (en Linux, los archivos de `/proc`; en macOS, `libproc` / `sysctl`; en Windows, Tool Help y PSAPI con sus contadores). Esa es la fuente de todos los datos. Nada de datos ya calculados. El porcentaje de CPU, los totales y el resumen se calculan en el código.
Prohibido usar librerías que los devuelvan resueltos: `gopsutil` (Go), `sysinfo`, `procfs` o `heim` (Rust), `libgtop` o `libprocps` (C/C++), o sus equivalentes en otros sistemas.
Nada de programas externos. El programa no ejecuta `ps`, `top`, `tasklist`, `wmic`, PowerShell, `cmd` ni ningún otro programa, no lee su salida, ni los usa para enviar señales.